

SDN Controller and Network-Control Applications

1 Overview

The SDN control plane is broadly divided into two components: the **SDN Controller** and the **SDN Network-Control Applications**. The controller provides a generic set of capabilities that allow applications to monitor and control the network's data-plane devices.

2 The Three Layers of an SDN Controller

A generic SDN controller's functionality is organized into three distinct layers:

1. Communication Layer: Southbound Interface

This layer handles the communication between the SDN controller and controlled network devices (switches, hosts, etc.).

- **Function:** Transferring information and observing local events (e.g., link up/down, heartbeats).
- **Southbound Interface:** The protocol used to "talk" to the devices.
- **Example: OpenFlow** is a dominant protocol implemented in most SDN controllers for this purpose.

2. Network-Wide State-Management Layer

The controller must maintain an accurate, up-to-date view of the network's state to make informed control decisions.

- **State Information:** Includes information about hosts, links, switches, and other SDN-controlled devices.
- **Flow Table Management:** Maintains copies of flow tables and associated statistics (counters) used by applications.
- **Logic:** Decisions like load balancing or firewalling are based on this global state.

3. Interface to Application Layer: Northbound Interface

This layer provides the API that network-control applications use to interact with the controller.

- **Function:** Allows applications to read/write network state and flow tables.
- **Events:** Applications can register to be notified of specific network events (state changes).
- **Northbound Interface:** Often implemented as a **RESTful API** (Request-Response interface).

3 Architecture and Scalability

Logically Centralized, Physically Distributed

While the controller appears as a single, monolithic service to the applications and switches, it is implemented as a **distributed system** for:

- **Fault Tolerance:** Ensuring service continues if a server fails.
- **High Availability:** Maintaining uptime for critical operations.
- **Performance:** Scalable performance for large networks.

Modern platforms like **OpenDaylight** and **ONOS** are designed with this distributed architecture in mind.

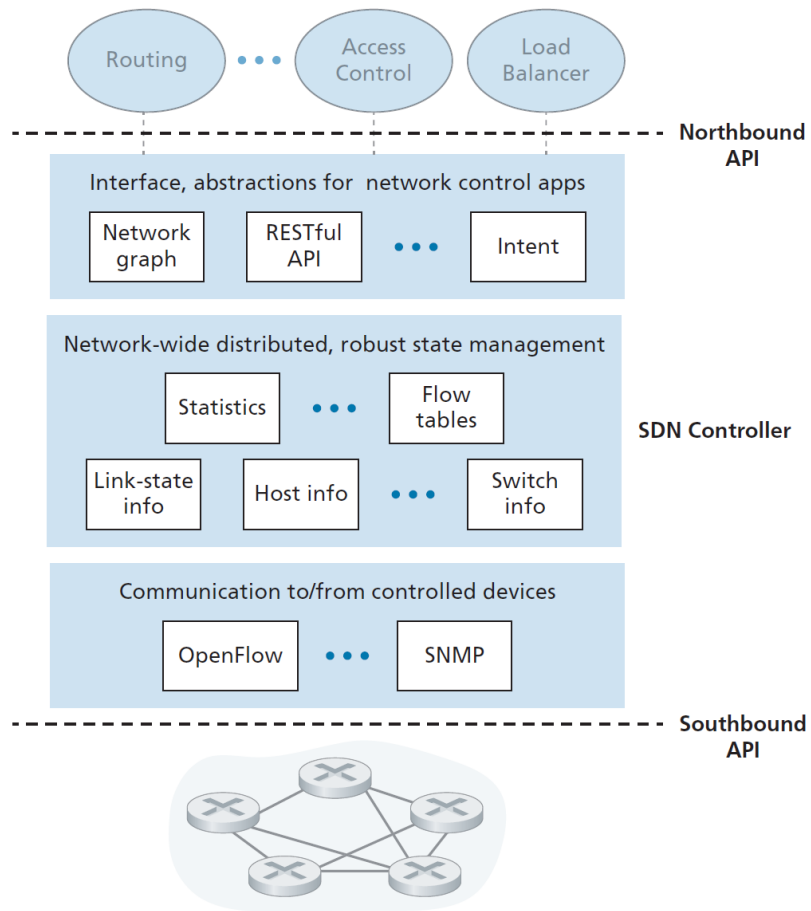


Figure 1: Components and layers of an SDN controller.

NOX, OpenDaylight, and ONOS

The structure of contemporary controllers closely mirrors the original NOX proposal (2008), evolving into today's robust, distributed platforms like OpenDaylight and ONOS.